Práctica N°1: Circuitos Combinacionales

Índice

1. Objetivos **2**

2. Ecuación **2**

3. Tabla de Verdad **2**

4. Descripción del circuito en VHDL **4**
4.1. Creación del proyecto 4
4.2. Creación del módulo VHDL 12
4.3. Explicación del código 12

5. Visualización del esquema RTL (Schematic) **13**

6. Asignación de pines de entrada y salida **15**
6.0.1. Acceso al panel de pines 15
6.0.2. Configuración del estándar lógico I/O STD 16
6.0.3. Asignación de pines físicos (Package Pins) 16
6.0.4. Verificación de pines en la documentación de la placa . 18

7. Proceso de síntesis, implementación y carga del bitstream en la placa Nexys A7 **19**
7.1. Synthesis 19
7.2. Implementation 20
7.3. Generación del Bitstream 21
7.4. Carga del Bitstream en la placa Nexys A7 22

8. Resultados en la Placa Nexys **23**

1. Objetivos

Objetivo general

Diseñar, describir en VHDL e implementar en una FPGA Nexys 4DDR un circuito combinacional a partir de una expresión lógica dada, utilizando el entorno de desarrollo Vivado y aplicando correctamente la lógica booleana.

Objetivos específicos

- Analizar y expresar algebraicamente la función lógica: $Q = (A \cdot \overline{B}) \ \ XOR \ \ \overline{(C + D)}$.
- Elaborar la tabla de verdad correspondiente a la función dada.
- Describir el circuito mediante código VHDL estructurado, verificando su sintaxis y lógica.
- Asignar correctamente las señales de entrada/salida a los pines de la FPGA.
- Sintetizar, implementar y programar el circuito en la tarjeta Nexys desde Vivado.
- Validar el comportamiento lógico del diseño en la placa, verificando que la salida coincida con lo esperado en la tabla de verdad.

2. Ecuación

La función lógica a implementar es:

$$Q = (A \cdot \overline{B}) \ \ XOR \ \ \overline{(C + D)}$$

3. Tabla de Verdad

La siguiente tabla muestra el desarrollo paso a paso de la función lógica:

$$Q = (A \cdot \overline{B}) \ \ XOR \ \ \overline{(C + D)}$$

A	B	C	D	$\overline{B}$	$A \cdot \overline{B}$	$C + D$	$\overline{(C + D)}$	Q
0	0	0	0	1	0	0	1	1
0	0	0	1	1	0	1	0	0
0	0	1	0	1	0	1	0	0
0	0	1	1	1	0	1	0	0
0	1	0	0	0	0	0	1	1
0	1	0	1	0	0	1	0	0
0	1	1	0	0	0	1	0	0
0	1	1	1	0	0	1	0	0
1	0	0	0	1	1	0	1	0
1	0	0	1	1	1	1	0	1
1	0	1	0	1	1	1	0	1
1	0	1	1	1	1	1	0	1
1	1	0	0	0	0	0	1	1
1	1	0	1	0	0	1	0	0
1	1	1	0	0	0	1	0	0
1	1	1	1	0	0	1	0	0

Cuadro 1: Evaluación paso a paso de la expresión lógica combinacional

Interpretación: Esta tabla detalla cada componente lógico intermedio involucrado en la evaluación de Q. La negación de B y la suma lógica $C + D$ permiten construir progresivamente la salida deseada. Esto no solo facilita la depuración y validación del circuito, sino que también entrena al estudiante a leer y descomponer funciones booleanas complejas, una habilidad clave en el diseño digital.

4. Descripción del circuito en VHDL

4.1. Creación del proyecto

Para comenzar con la implementación del circuito combinacional, se debe crear un nuevo proyecto en Vivado. El procedimiento es el siguiente:

1. Abrir Vivado y seleccionar **Create Project**.

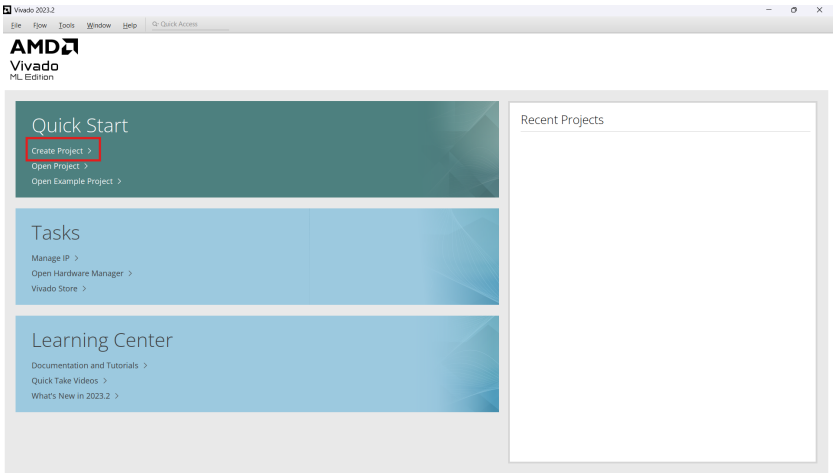


Figura 1: Ventana de Inicializacion de Vivado para crear el proyecto.

2. Asignar un nombre representativo al proyecto (por ejemplo, `P1circuitocombinacional`) y definir la ubicación donde se guardará.

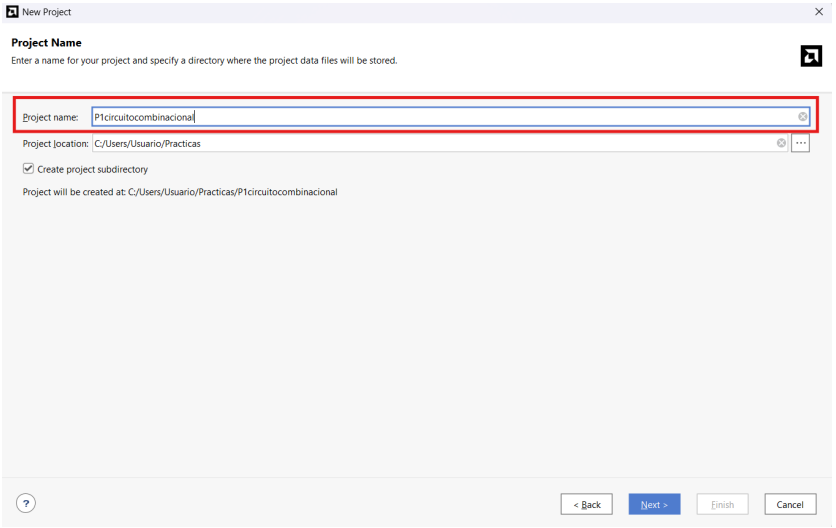


Figura 2: Nombre del proyecto.

3. En el asistente, seleccionar **RTL Project**.

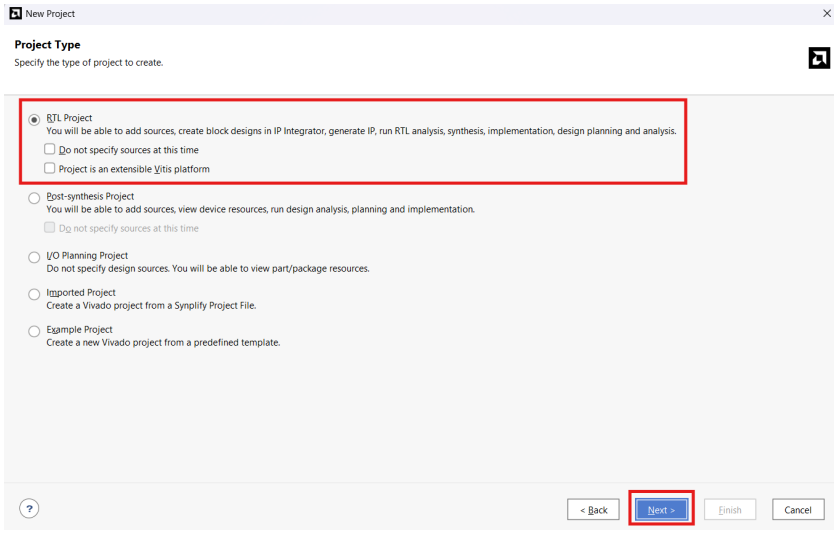


Figura 3: Tipo de proyecto creado.

4. A continuación, en el menú de recursos, debemos revisar las opciones de **Target Language:** y **Simulator Language:** ambas deben estar en la opción **VHDL**

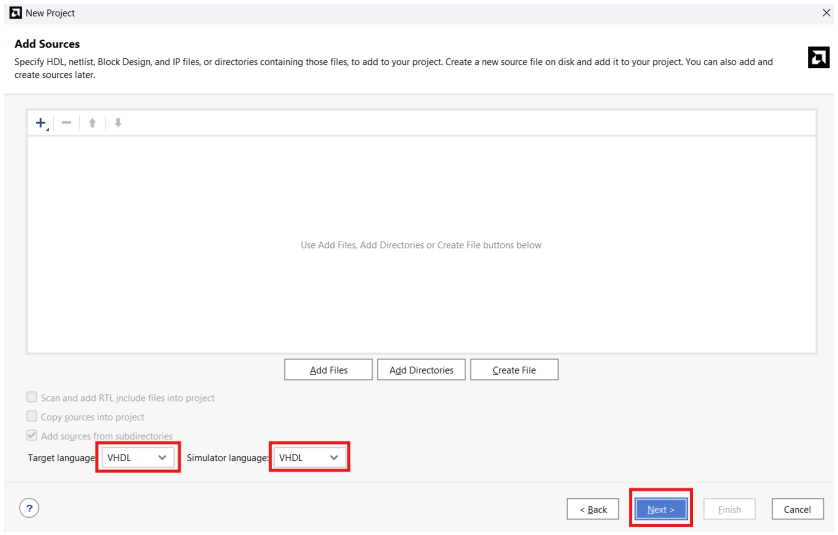


Figura 4: Recursos del proyecto.

5. El siguiente menú de configuraciones es el de restricciones, en este no vamos a configurar nada, dirigiéndonos al siguiente paso

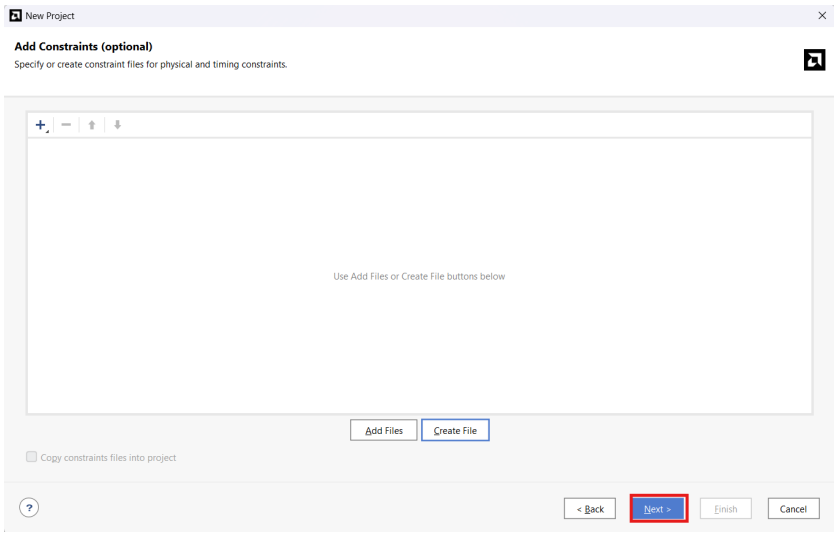


Figura 5: Restricciones del proyecto.

6. En la sección de selección de la tarjeta, buscar y elegir el modelo correspondiente. Para esta práctica se utiliza:

   `NEXYS DDR4`

   Esta corresponde a la FPGA que integra la tarjeta Nexys DDR4. Para ello nos dirigimos al menú de Boards, en este utilizando el buscador, se introduce el código de la placa. En caso que no este descargado, la comuna "status"se encontrara con un símbolo de descarga.

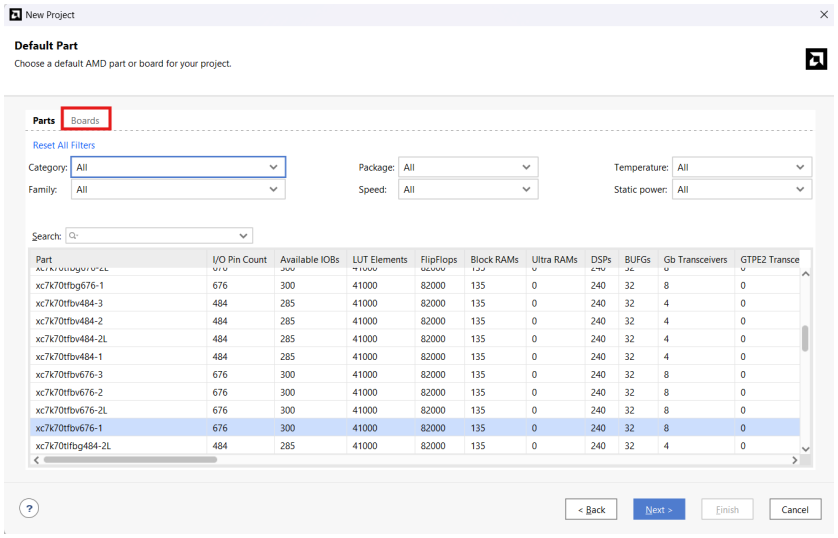


Figura 6: Acceso al menú de placas disponibles.

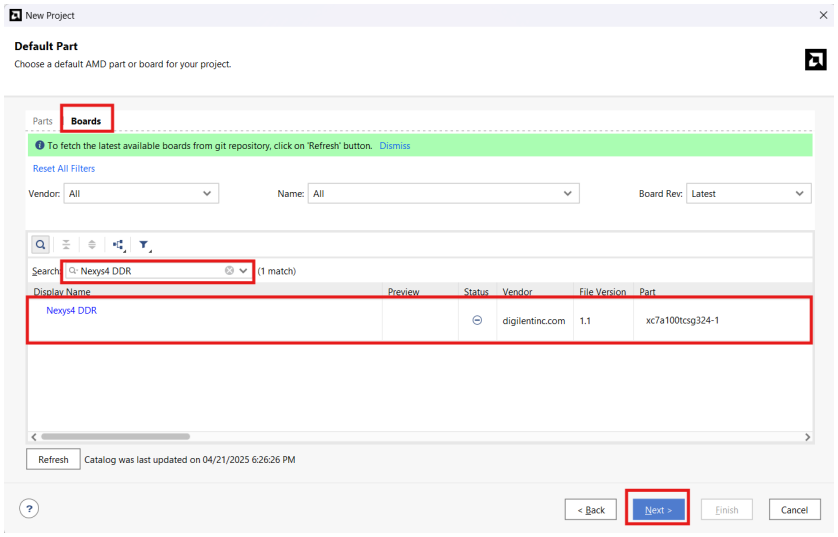


Figura 7: Seleccion de la placa.

7. Una vez terminada esta configuración, se debe finalizar la creación del proyecto, con la opción **Finish**, con ello completamos este proceso.

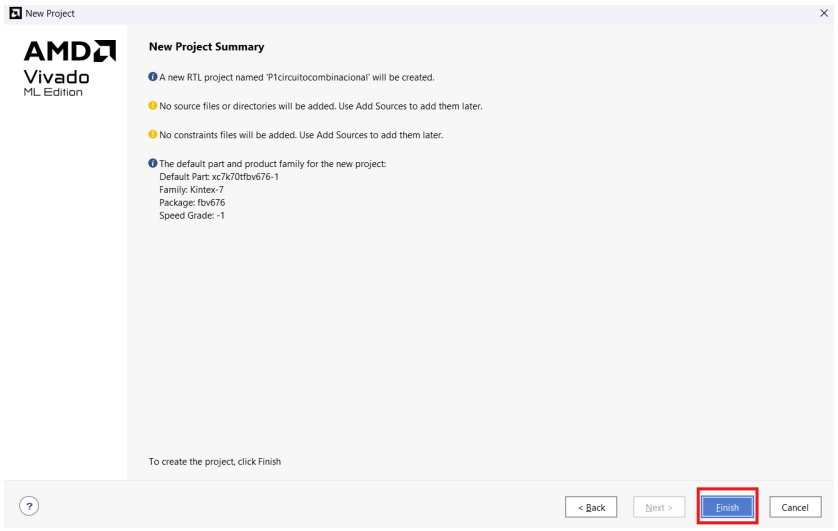


Figura 8: Etapa final de la creación del proyecto.

8. Con el proyecto creado, el siguiente paso es crear el archivo **.vhd** en el cual se programará el codigo con la logica requerida, para ello se debe utilizar el simbolo '+' ubicado en el menú de recursos.

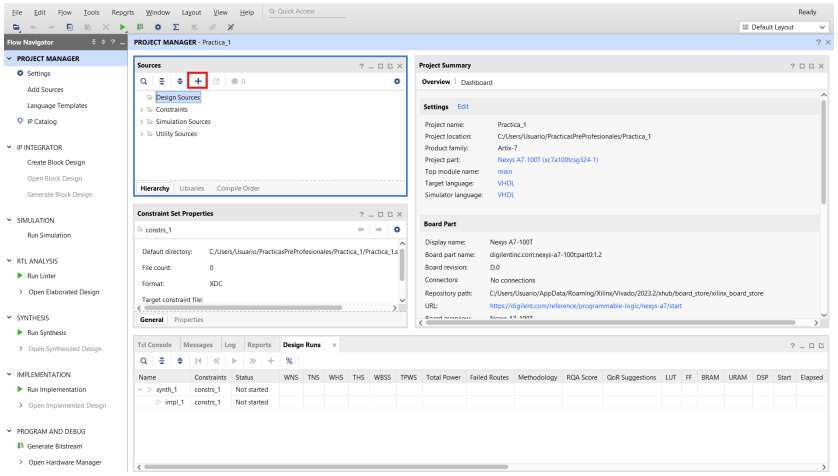


Figura 9: Ubicacion del botón para la creacion del archivo **.vhd**.

9. Se verifica la opción 'Add or create design sources' con esto se continua para la creación del archivo

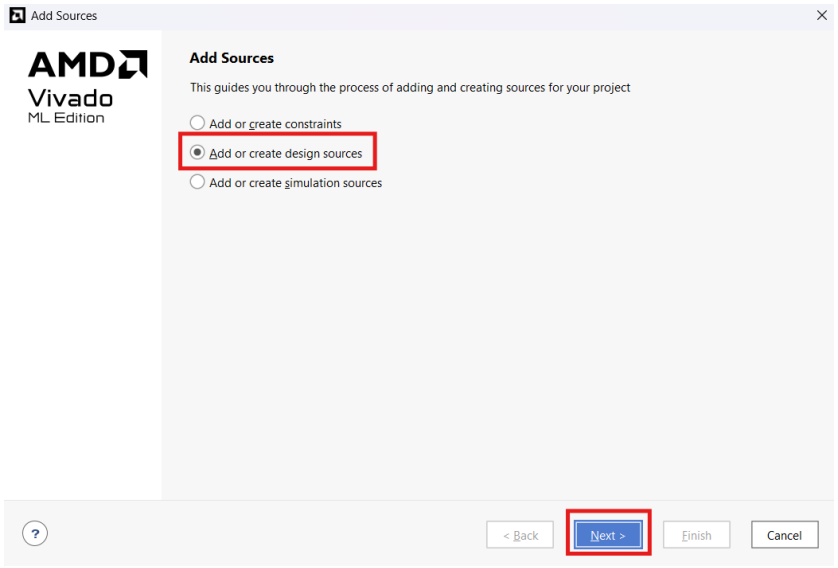


Figura 10: Verificación de tipo de recurso a crear.

10. En el siguiente menú se debe seleccionar 'Create File' con ello se abrirá un panel donde se coloca el nombre del archivo a crear, se recomienda el nombre **main** debido a que se suele usar el nombre main para el archivo principal por analogía con otros lenguajes de programación (como C/C++ o Python), donde main representa el punto de entrada del programa. Aunque en VHDL no existe una función main como tal, este nombre permite identificar rápidamente el módulo que describe el comportamiento principal del circuito.

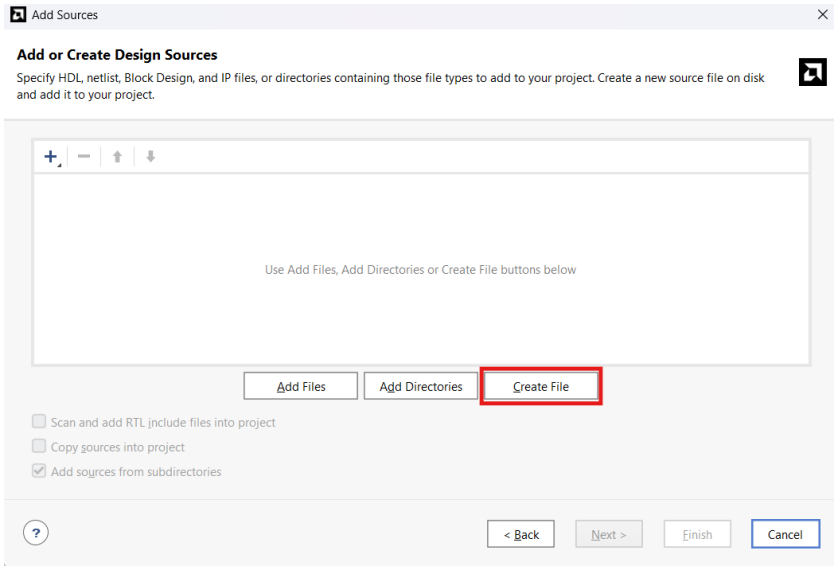


Figura 11: Manú para la creacion de archivos.

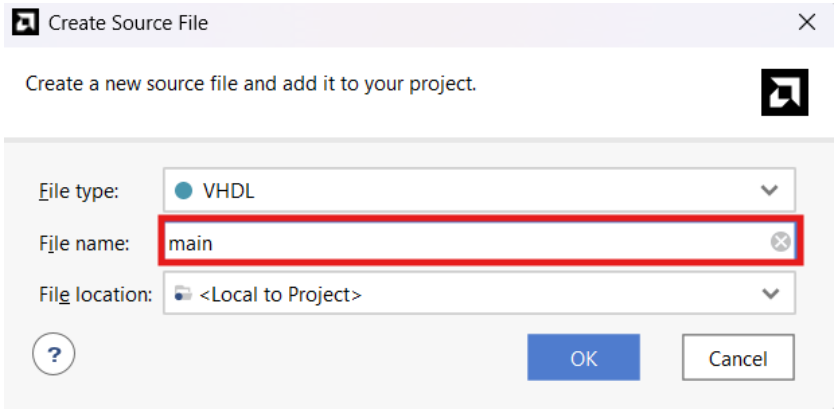


Figura 12: Nombre del archivo .vhd creado.

11. De ser necesario se crean la cantidad de archivos necesarios para cada proyecto, en este caso no es necesario debido a la simpleza del mismo. Una vez finalizado la creación de archivos se finaliza, creando un panel donde se debe En caso de crear un nuevo archivo, especificar el nombre de la arquitectura y describir los puertos, este último paso no es necesario, se los puede definir por código posteriormente.

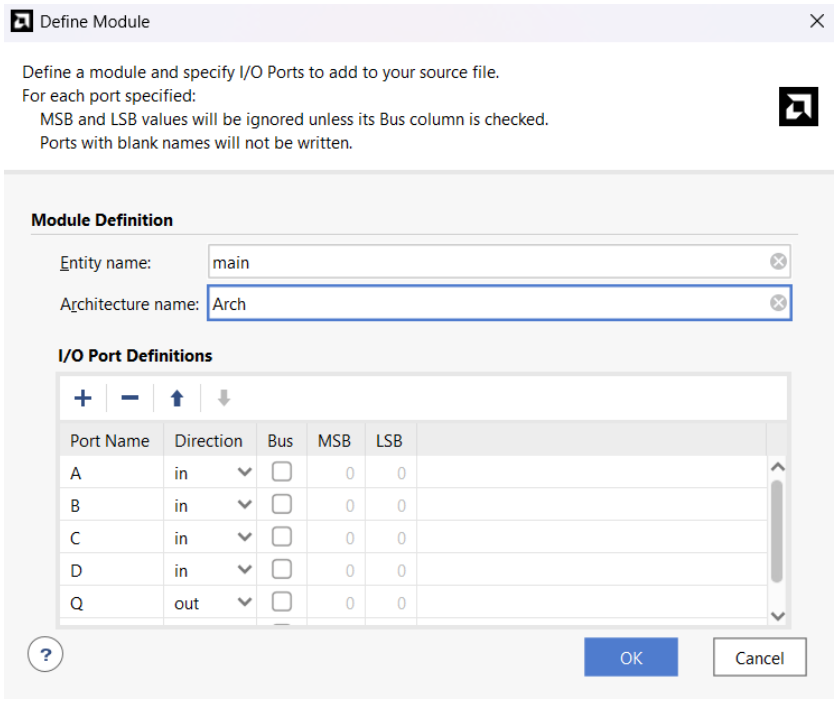


Figura 13: Panel para definir entradas y salidas.

12. Finalmente para entrar al editor de texto del código en el apartado de **Design Sources** se debe hacer doble click sobre el archivo creado, en el panel derecho se abre el archivo, de haber especificado las entradas y salidas, estas se crean de manera automática, de lo contrario se deben colocar manualmente

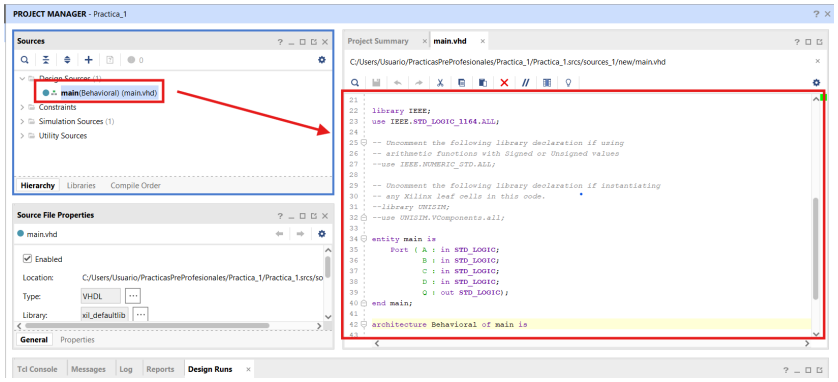


Figura 14: Código main creado.

4.2. Creación del módulo VHDL

Una vez creado el archivo `.vhd` donde se describirá el comportamiento del circuito se debe comenzar a programar. A continuación se presenta el código VHDL comentado línea por línea, con el objetivo de facilitar su comprensión:

```
1  -- Importacion de bibliotecas estandar
2  library IEEE;
3  use IEEE.STD_LOGIC_1164.ALL;
4
5  -- Declaracion de la entidad
6  -- La entidad representa la interfaz del circuito: sus
       entradas y salidas
7  entity P1circuitocombinacional is
8      port (
9          A : in std_logic;  -- Entrada A
10         B : in std_logic;  -- Entrada B
11         C : in std_logic;  -- Entrada C
12         D : in std_logic;  -- Entrada D
13         Q : out std_logic  -- Salida Q
14     );
15 end P1circuitocombinacional;
16
17 -- Arquitectura del circuito
18 -- Define el comportamiento funcional del circuito
       descrito en la entidad
19 architecture Arch of P1circuitocombinacional is
20 begin
21     -- La salida Q se define como:
22     -- (A AND NOT B) XOR NOT(C OR D)
23     Q <= (A and (not B)) xor (not (C or D));
24 end Arch;
```

Listing 1: Descripción VHDL del circuito combinacional

4.3. Explicación del código

- **Librerías:** Se importan las librerías estándar del paquete IEEE, particularmente `STD_LOGIC_1164`, que permite el uso de tipos de datos como `std_logic`, necesarios para describir señales digitales de uno y cero (y otros estados como 'Z' o 'X').

- **Entidad:** La entidad `P1circuitocombinacional` define los puertos de entrada y salida del módulo. En este caso, se tienen cuatro entradas

(`A, B, C, D`) y una salida (`Q`), todas de tipo `std_logic`.

- **Dirección de los puertos:** Se utiliza `in` para definir entradas, y `out` para definir salidas. Esto es fundamental para que el compilador del hardware entienda la dirección del flujo de datos.
- **Arquitectura:** Bajo el estilo `Behavioral`, se describe el comportamiento lógico del circuito. Aquí se asigna directamente a la salida `Q` la expresión booleana requerida, utilizando operadores lógicos estándar de VHDL:
  - `and` → conjunción lógica (AND)
  - `or` → disyunción lógica (OR)
  - `not` → negación lógica (NOT)
  - `xor` → disyunción exclusiva (XOR)
- **Asignación concurrente:** La línea que define la salida `Q` se ejecuta en paralelo con el resto del hardware (no es secuencial), como ocurre en los lenguajes de descripción de hardware.

5. Visualización del esquema RTL (Schematic)

Una vez completada y guardada la descripción en VHDL, podemos generar automáticamente un esquema lógico del circuito a partir del código fuente. Este esquema nos permite visualizar gráficamente cómo se conectan las señales y cómo se interpreta la arquitectura descrita.

Acceso al análisis RTL

Para ello, realizamos los siguientes pasos dentro del entorno de desarrollo Vivado:

1. En el panel de flujo de diseño (Flow Navigator), ubicamos la sección **RTL Analysis**.
2. Damos clic en la opción **Open Elaborated Design**.
3. Una vez se abra el diseño elaborado, nos dirigimos al menú superior y seleccionamos la opción **Schematic**.

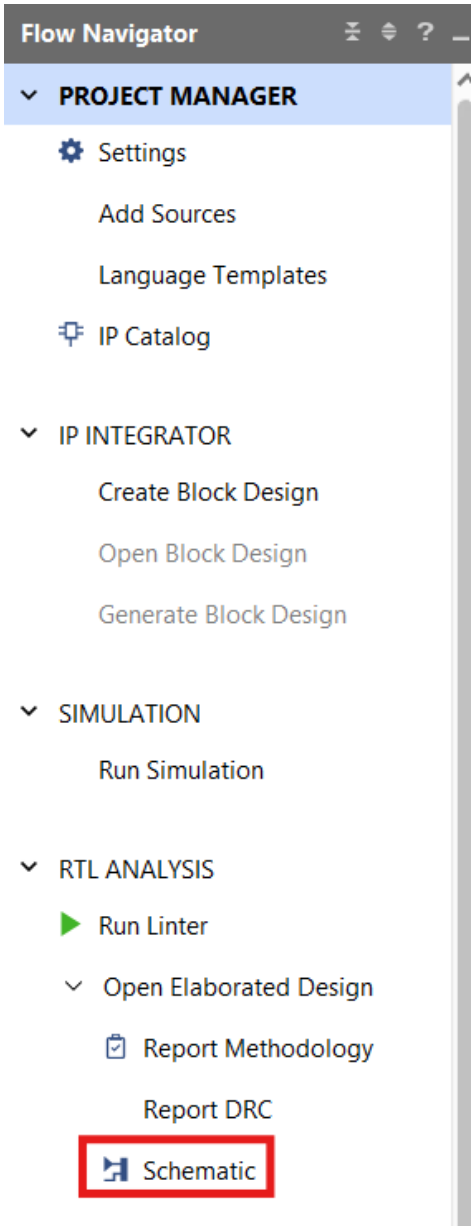


Figura 15: Ubicación de la opción `Open Elaborated Design` en Vivado.

Interpretación del diagrama generado

Después de ejecutar estos pasos, Vivado genera automáticamente un esquema que representa la interconexión de los elementos descritos en el archivo VHDL.

- Este diagrama muestra los **puertos de entrada y salida** declarados

en la entidad.

- Se visualizan también los operadores lógicos y las compuertas que conforman la lógica del circuito combinacional.
- Es una herramienta útil para verificar visualmente si el comportamiento programado coincide con la lógica esperada.

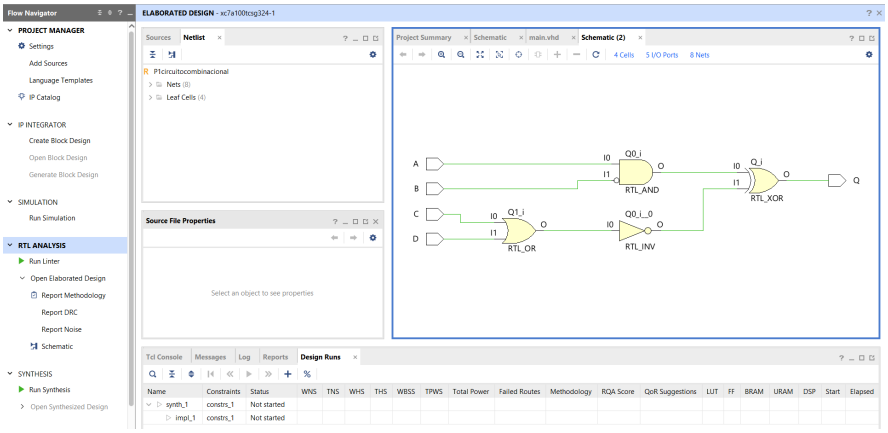


Figura 16: Esquemático generado automáticamente por Vivado a partir del código VHDL.

> **Nota:** Es recomendable verificar que las señales estén correctamente conectadas y que la lógica visual refleje la función booleana implementada, como ayuda complementaria a la simulación.

6. Asignación de pines de entrada y salida

Una vez verificado el esquema del circuito, el siguiente paso en el flujo de diseño es la asignación física de los pines de entrada y salida del FPGA. Esta configuración es esencial para vincular correctamente las señales lógicas del código VHDL con los dispositivos físicos de la tarjeta Nexys A7.

6.0.1. Acceso al panel de pines

Desde el diseño elaborado en el esquema RTL:

1. Damos clic en la opción **I/O Ports**, ubicada en el margen izquierdo de la vista del esquema.

2. Se abrirá un panel en la parte inferior del entorno Vivado, donde se listan automáticamente las señales declaradas en la entidad del código VHDL.

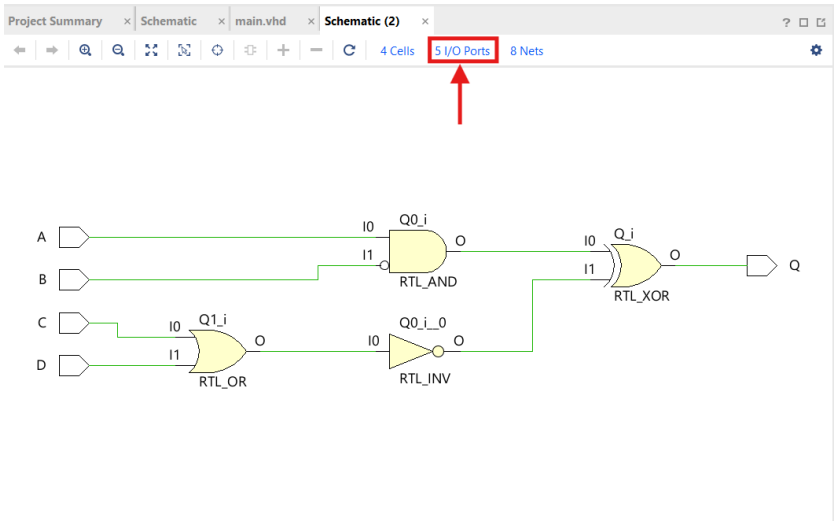


Figura 17: Acceso al panel de asignación de puertos (I/O Ports).

6.0.2. Configuración del estándar lógico I/O STD

En la columna `I/O Std` del panel de pines, se debe configurar el estándar de voltaje para cada señal. Seleccionamos la opción:

LVCMOS33

Este estándar se refiere a la lógica de tipo **Low Voltage CMOS a 3.3V**, que es el nivel lógico compatible con la mayoría de los periféricos de la placa Nexys A7. Esta configuración asegura que los niveles de voltaje de las señales de entrada y salida se ajusten adecuadamente al hardware físico.

> **Nota:** Todos los pines deben tener el mismo estándar lógico, ya que la placa opera con lógica de 3.3V. Usar otro estándar podría causar mal funcionamiento o incluso dañar el FPGA.

6.0.3. Asignación de pines físicos (Package Pins)

Posteriormente, en la columna `Package Pin`, asignamos manualmente cada una de las señales a su correspondiente pin físico de la tarjeta. En este ejemplo, se realiza la siguiente correspondencia:

Señal VHDL	Pin FPGA	Función en la tarjeta
A	J15	Switch 0
B	L16	Switch 1
C	M13	Switch 2
D	R15	Switch 3
Q	H17	LED 0

Cuadro 2: Asignación de señales a pines físicos en la placa Nexys A7.

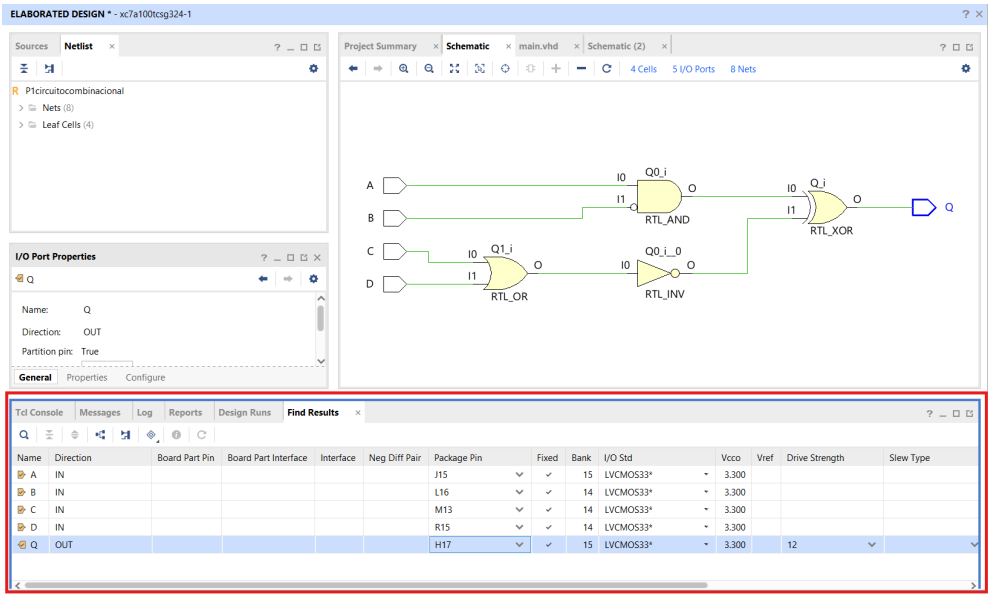


Figura 18: Vista de asignación de pines en Vivado.

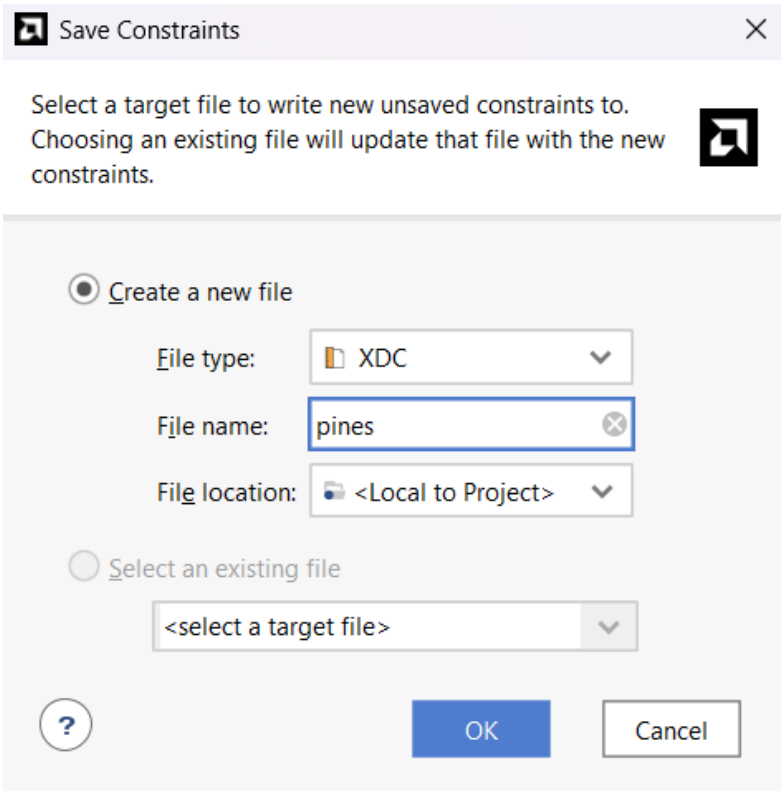


Figura 19: GUardado del archivo de asignación de pines en Vivado.

6.0.4. Verificación de pines en la documentación de la placa

Aunque algunos nombres de pines están serigrafiados directamente sobre la placa Nexys A7 (cerca de los switches o LEDs), la verificación más precisa se realiza consultando la **hoja técnica oficial** (User Guide o Reference Manual) proporcionada por Digilent. En este documento se encuentra el listado completo de los pines y su correspondencia con los periféricos de la placa.

> **Recomendación:** Siempre verificar la asignación de pines con el archivo `.xdc` de referencia que proporciona Digilent para evitar errores de conexión que puedan dañar el FPGA o hacer que el diseño no funcione correctamente.

7. Proceso de síntesis, implementación y carga del bitstream en la placa Nexys A7

Luego de definir el diseño lógico en VHDL y realizar la asignación de pines, el siguiente paso es llevar a cabo la síntesis e implementación del circuito, seguido por la generación y carga del archivo de configuración (`.bit`) en la FPGA.

7.1. Synthesis

El primer paso en el flujo de Vivado es la **síntesis**.

- Se accede a través del menú izquierdo de Vivado, haciendo clic en **Run Synthesis**.

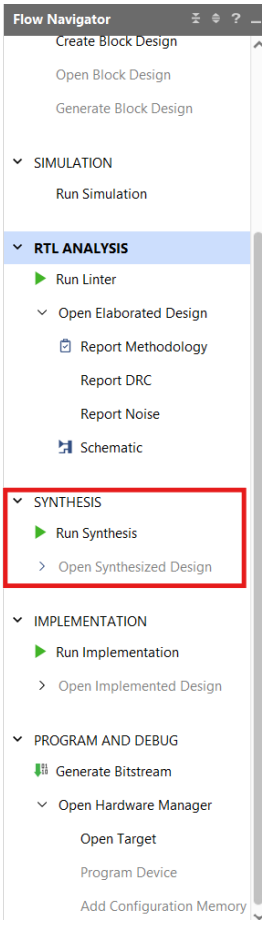


Figura 20: Opción *Run Synthesis* en el flujo de Vivado.

¿Qué es la síntesis? La síntesis es el proceso mediante el cual el código VHDL es traducido a una representación lógica a nivel de compuertas. Esta etapa convierte las descripciones de alto nivel del comportamiento digital en una red lógica compuesta por LUTs (Look-Up Tables), registros, multiplexores, entre otros.

Importante: Es fundamental esperar a que la síntesis se complete satisfactoriamente. Al finalizar, se abrirá un panel con varias opciones. En este caso, se debe seleccionar la opción:

Run Implementation

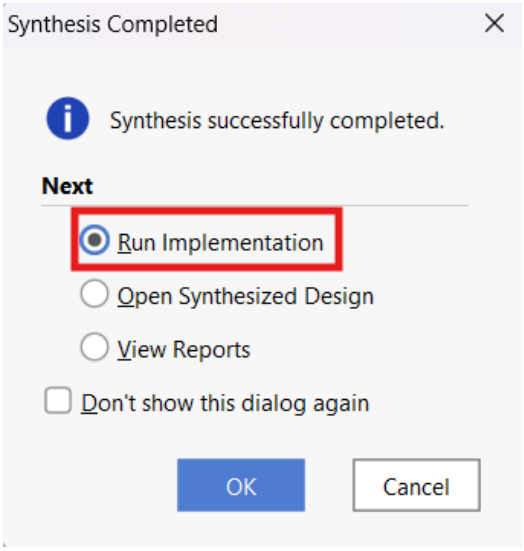


Figura 21: Selección de *Run Implementation* tras la síntesis.

7.2. Implementation

La implementación, también llamada *place and route*, mapea la lógica sintetizada sobre los recursos físicos de la FPGA, ubicando cada celda lógica, conectándola mediante las rutas disponibles y resolviendo restricciones temporales.

- Se ejecuta automáticamente si se seleccionó **Run Implementation** tras la síntesis, o manualmente desde el menú izquierdo.

¿Por qué es necesaria? Este paso garantiza que la lógica se ajuste físicamente a la arquitectura del FPGA, considerando las limitaciones de espacio, temporización y conectividad. Sin esta etapa, no se puede generar un archivo válido de configuración.

Al finalizar la implementación, se abrirá nuevamente un menú. Esta vez, se debe seleccionar la opción:

Generate Bitstream

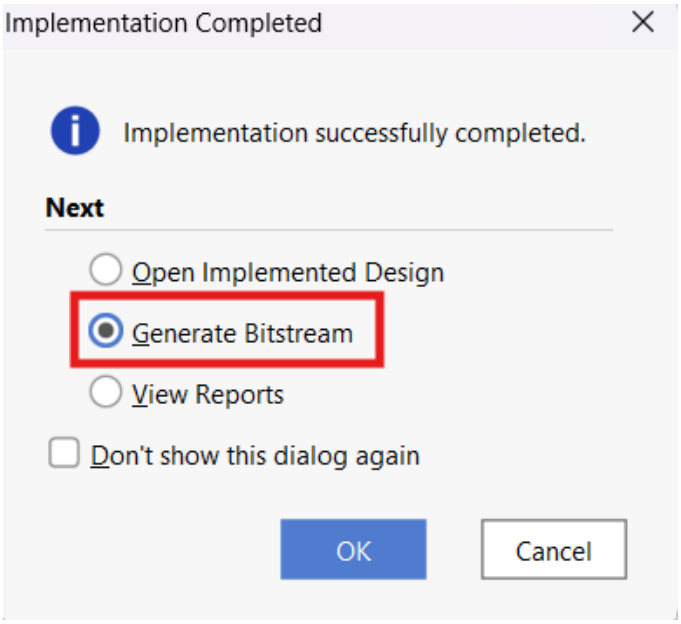


Figura 22: Selección de *Generate Bitstream* al finalizar la implementación.

7.3. Generación del Bitstream

El archivo `.bit` contiene la información binaria necesaria para programar la FPGA con el diseño implementado. Este archivo configura todos los recursos lógicos internos y define el comportamiento del circuito.

- Una vez seleccionada la opción **Generate Bitstream**, Vivado realiza un empaquetado final.
- Al terminar, se muestra un cuadro de confirmación, el cual se puede cerrar sin realizar cambios adicionales.

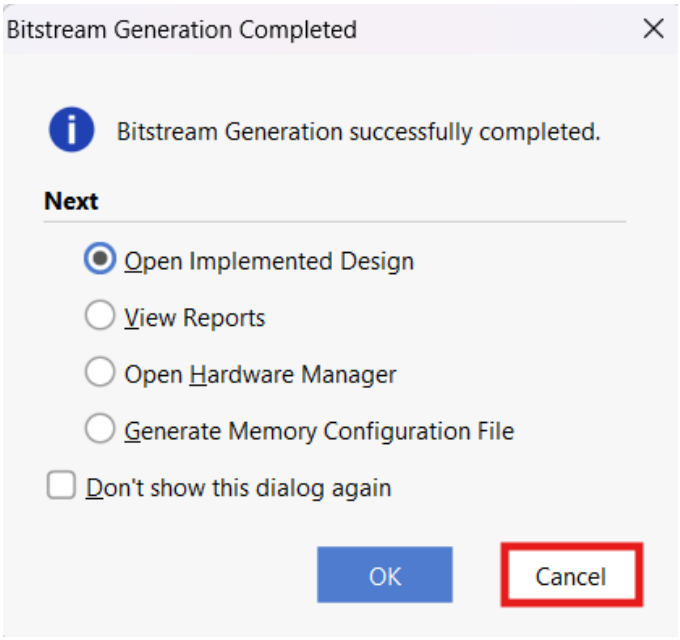


Figura 23: Notificación de bitstream generado.

7.4. Carga del Bitstream en la placa Nexys A7

Con el archivo `.bit` generado y la placa Nexys A7 conectada vía USB, se procede a cargar el diseño en el dispositivo FPGA.

1. Desde el menú izquierdo, seleccionar **Open Hardware Manager**.
2. Luego, hacer clic en **Open Target** → **Auto Connect**.
3. Si la placa está correctamente conectada, aparecerá en la lista de dispositivos disponibles.
4. Hacer clic derecho sobre el dispositivo detectado y seleccionar **Program Device**.
5. Elegir el archivo `.bit` generado previamente.
6. Finalmente, presionar **Program**.

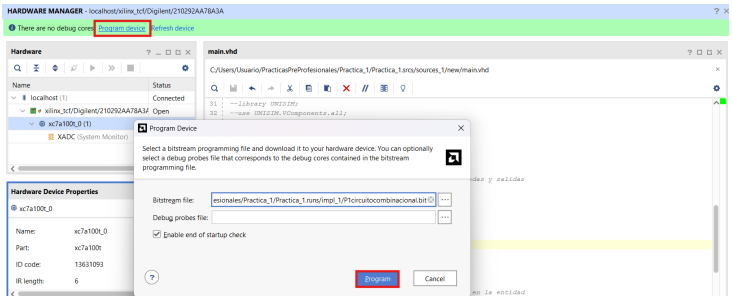


Figura 24: Selección del bitstream y programación de la FPGA.

Resultado: Una vez cargado el bitstream, la FPGA comenzará a ejecutar el diseño configurado. En este caso, los switches conectados a las señales de entrada A, B, C y D afectarán directamente al LED Q, lo cual permite verificar el correcto funcionamiento del diseño implementado.

8. Resultados en la Placa Nexys

Una vez cargado correctamente el archivo `.bit` en la tarjeta de desarrollo Nexys, se puede verificar el funcionamiento del circuito combinacional en tiempo real, utilizando los interruptores (switches) físicos para las entradas A, B, C y D, y observando el estado del LED correspondiente a la salida Q.

Ejemplos de Combinaciones de Entrada

En la siguiente tabla se muestran algunos ejemplos específicos de combinaciones aplicadas directamente en la placa, junto con el resultado observado en el LED de salida:

A	B	C	D	Expresión Q	LED Encendido
0	0	0	0	$(0 \cdot 1) \ XOR \ (0+0)' = 0 \ XOR \ 1 = 1$	Sí
1	0	1	1	$(1 \cdot 1) \ XOR \ (1+1)' = 1 \ XOR \ 0 = 1$	Sí
1	1	0	0	$(1 \cdot 0) \ XOR \ (0+0)' = 0 \ XOR \ 1 = 1$	Sí
1	1	1	1	$(1 \cdot 0) \ XOR \ (1+1)' = 0 \ XOR \ 0 = 0$	No
0	0	1	0	$(0 \cdot 1) \ XOR \ (1+0)' = 0 \ XOR \ 0 = 0$	No

Cuadro 3: Resultados observados directamente en la placa Nexys

Captura Fotográfica del Comportamiento Real

A continuación se muestran fotografías capturadas durante la ejecución del circuito en la placa Nexys, evidenciando los valores de entrada aplicados mediante los interruptores y el estado del LED indicador de la salida:

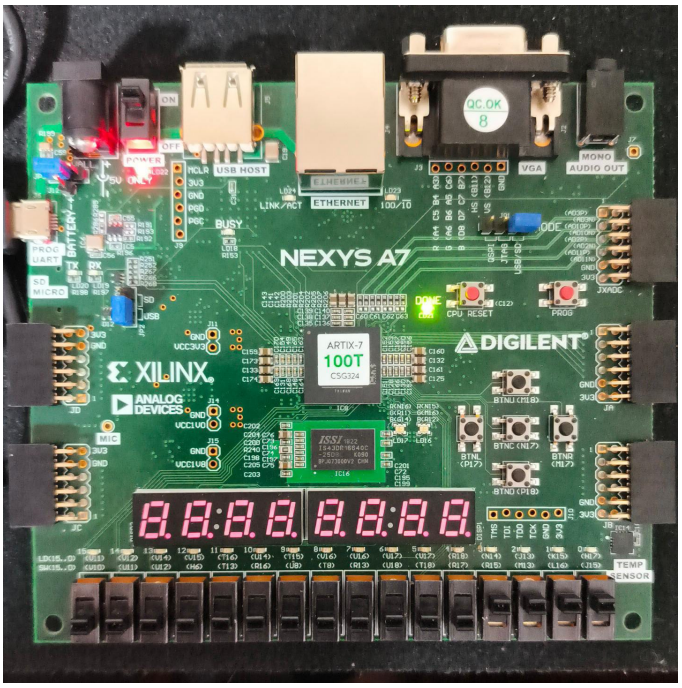


Figura 25: Combinación A=1, B=1, C=1, D=1. LED apagado (Q=0)

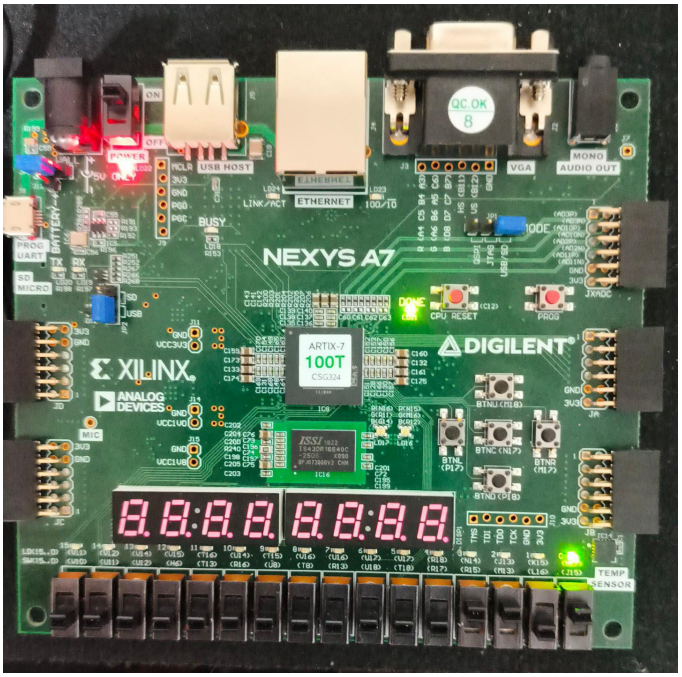


Figura 26: Combinación A=1, B=0, C=1, D=1. LED encendido (Q=1)

Conclusión del Comportamiento Físico

Se puede concluir que la implementación física en la tarjeta Nexys responde fielmente al diseño lógico planteado en VHDL. Las combinaciones lógicas dadas a través de los switches activan o desactivan el LED de salida conforme a la ecuación implementada:

$$Q = (A \cdot \overline{B}) \quad XOR \quad \overline{(C + D)}$$

Esto valida que el proceso de síntesis, implementación y asignación de pines fue realizado correctamente.